BM GAME API - VOLUME 5

bm game API, volume 5: `tools`, projects and resources

Cartridges, projects and resource files, the functions the development tools use, and how to share assets between games.

Contents

The cartridge file 4

What it does 4

Why it matters 4

Good to know 4

Caveats 5

Try it, step by step 5

From the PC 5

The sections of a cartridge 6

The section types 6

Good to know 7

The API for tools 8

Games and projects in the API 8

Keyboard and files 8

Reading and writing cartridges 8

Audio for the editors 9

3D models 10

Running, debugging and moving between tools 10

The assistant 11

Resource files and sharing assets 15

The file types 15

What it does 15

Good to know 15

Caveats 15

What integration does 16

Try it, step by step: move a model from one game to another 16

The other commands 17

The project layout 18

What it does 18

Good to know 18

Caveats 18
Try it, step by step: migrate a game to the common layout 18

Screenshots for developers 20
What it does 20
Good to know 20
Caveats 20
Try it, step by step 21
For a manual 21

Open points 22

README

What this is. Volume 5 of the bm game API. It is for you if you build tools for the console, or if you pack and share games.

How to use it. The first two chapters say what a cartridge file is and what is inside. The third lists every function the tools use. The last three are guided: sharing assets, the project layout, screenshots.

What is missing. Every gap is marked with a highlighted TO FILL tag; nothing is invented.

The cartridge file

A game on bm is one file. The same container comes in three shapes, and what matters is which tools may change it.

Extension	What it is	Can bm's tools change it?
`.bm`	a **game**: it is in the Games tab of the menu	No. Tools read it and take its code and assets
`.b16`	a **game** too: the limited-resource edition for handhelds	No, as for `.bm`
`.bme`	a **project**: the same container, the one tools edit	Yes

What it does

A project is a `.bme` with bit 0 of the u16 at offset 18 of the header set. A `.b16` is marked with bit 1 of the same u16. Nothing else in the container changes. A project makes its game with a build; a game makes an editable copy of itself.

Why it matters

Your game is never overwritten by a tool by accident. You edit the project, and you build the game when it is ready.

Good to know

- A tool that saves a game asks first: *Save an editable copy: NAME.BME?* The copy goes in `/bme` (`/bme/NAME.BME`, or `NAME1.BME` if that name is taken). A `.bme` found next to its game on an older card is still read, and saved where it is.
- In the menu, *Make an editable copy* is in a game's options. The project is not listed in the menu tabs: open it from the SDK with *Open* (**Ctrl+O**).
- *Build the game (.bm)* is in a project's options. In the SDK the key is **Ctrl+B**.
- A build of a project in `/bme` writes the game in the games folder (`/carts` on the Pi, `/bm` on the RGB30); a project next to its game builds next to it. The game has the project's name stored at offset 104 of the header. The next build replaces that game and only that one. A game of the same name that it did not build stays, and the new one takes a digit (`PONG1.BM`).
- The `.bm` and the `.b16` are separate editions. A `.b16` never replaces the `.bm`, and the `.bm` sources are never overwritten to make a `.b16`.

- A `.b16` file has no size limit of its own: the build tools no longer refuse a large file. The Market takes up to 100 MiB a file. Its screen is 360x360 (or 720x720, the profile says), the same on the Pi and on the RGB30.
- Other cartridges write only `.bm` and `.bme` files in `/carts`, and only ones they made in the same run.

Caveats

- [TO FILL: status of the `.b16` profile. `docs/B16.md` says the profile is decided but not yet implemented, and §3 describes a different `B16CART` container that §0 replaces with the `.bm` container; `src/bm/project.h` and `scripts/mkbm.py` already mark and export `.b16` files (bit 1 at offset 18, 8 MiB check). Say which one is current.]
- The SDK's *Build .bm* and *Export .b16* entries call `bm_build_b16`; **Ctrl+Shift+B** in the bm SDK always exports a `.b16`.

Try it, step by step

1. On the console, open the Games tab and select a game.
2. Press **X** on the RGB30 pad (**C** on the Pi) to open its options and choose *Make an editable copy*.
3. Open the Dev tab and start the **bm SDK**. The tab does not list projects.
4. In the SDK press **Ctrl+O** and choose the project.
5. In the SDK press **Ctrl+B** (*Build the game .bm*). The game appears in the Games tab.

[TO FILL: shots/api-tools-editable-copy.png]

TRY IT

Build the same project twice. The second build replaces the first game: you do not get a second file.

From the PC

`mkbm.py` writes the kind of file from the extension of `-o`: `.bm`, `.bme` or `.b16`.

COPY AND PASTE

```
python3 scripts/mkbm.py -o ball.bme --lua carts/ball/main.lua --title "Ball" --author "me"
python3 scripts/mkbm.py --from ball.bm --b16
```

BEWARE

Nothing refuses a large `.b16` at build time. With `--from` and `--b16` and no `-o`, the file is written next to the source with the `.b16` extension.

The sections of a cartridge

A cartridge is a 128-byte header, a section table and the section bodies. Everything is little endian.

Offset	Content
0	magic `BMCART` and two zero bytes (`BM33CART` in old files, still read)
8	u16 version (1)
10	u16 header size (128)
12	u16 width, u16 height: 640x360, 480x270, 320x180, 256x256, 360x360
16	u8 pixel format (1 = RGB565)
17	u8 section count
18	u16 flags: bit 0 project, bit 1 `.b16`
20	u32 CRC-32 of everything after the header
24	char[48] title
72	char[32] author
104	char[16] the project a game was built from; zero up to 128
128	the section table: for each section u32 type, u32 offset, u32 size, u32 reserved

The section types

Type	Name	What it holds
1	LUA	the main script, UTF-8 source
2	SHEET	the sprite sheet: u16 w, u16 h, then RGBA8888 pixels (alpha below 128 is transparent)
3	MAP	u16 w, u16 h, then u16 cells (sprite index, 0 = empty)
4	COVER	the picture of the cartridge in the menu (`mkbm.py` makes it 88x88); first in the file
5	SHEET8	the sheet with at most 256 colours and runs, much smaller; a cartridge has SHEET or SHEET8, not both
6	AUDIO	the sound bank (`BMAU`): sounds, effects, patterns, songs; played by `sfx()` and `music()`

Type	Name	What it holds
8	MESH	3D models, 1 to 256, with the texture inset
9	ANIM	skeletons and animations of the MESH models
10	INFO	`key: value` text of at most 16 KiB about the file and its parts
11	SPRITES	named zones of the sheet (1 to 1024), with frames and speed
12	LAYERS	the other layers of the map (2 to 8, the MAP section is layer 1)
13	FLAGS	the 8 flags of each 8x8 cell of the sheet
14	BOXES	hitboxes and hurtboxes of the frames of the SPRITES zones (1 to 4096)

Good to know

- Types 6 and 7 were MESH and ANIM in the first bm Studio files. A type 6 section that is not a sound bank is still read as MESH, a type 7 as ANIM. They are written as 8 and 9.
- Graphics are stored independently of the screen format and converted when the cartridge is loaded.
- A section the kernel does not know is kept when a tool rewrites the file.
- Tile flags used by bmlib: 0 solid, 1 platform, 2 ladder, 3 water, 4 hurts.

The API for tools

Part of the game API. These functions are meant for the development tools (the editors of the suite). Games do not need them, and the design in `docs/B16.md` says the `.b16` profile leaves the `cart_*` functions and `ls` out of the sandbox; the console does not enforce it yet.

Games and projects in the API

`cart_save`, `cart_write` and `cart_put_audio` return `true` **and the file written** when it is not the one you asked for. This happens when you save a game: the tool writes the editable copy and goes on with that file. A new `.bm` becomes a project (`"/carts/NEW.BM"` becomes `/carts/NEW.BME`). A project makes its game with `cart_build`.

Keyboard and files

Function	Description
`keyheld(name)`	`true` while key `name` is held: `"f1"` to `"f12"`, `"tab"`, `"space"`, `"enter"`, `"esc"`
`keyp()`	the next key typed: a character (`"a"`, `"\n"` Enter, `"\b"` Backspace, `"\t"`), a name (`"up"`, `"down"`, `"left"`, `"right"`, `"home"`, `"end"`, `"pgup"`, `"pgdn"`, `"del"`, `"esc"`, `"f1"` to `"f10"`), `"^s"` for Ctrl+S, `"^S"` for Ctrl+Shift+S (also `"^i"`, `"^m"`); `nil` if none. F11 and F12 are the system's and never come. From the first call the keyboard types and no longer works as a gamepad for `btn()`
`keyhelp(list, [title])`	shows the cartridge's keys under the system's while **F12** is held. `list` is `{ {"keys", "what they do"}, "subtitle", ... }`, keys as in `prompt()` (lower case the keyboard, upper case the pad). Returns how many entries name a key the system keeps for itself (F11, F12, Ctrl+Esc, Ctrl+Shift+Esc): they are shown in red and in the log. `keyhelp(nil)` takes it away
`ls([folder])`	the files of the SD card: `{ {name=, size=, dir=}, ... }`

Reading and writing cartridges

Function	Description
`cart_load(path)`	opens a `.bm`: its sprite sheet (with flags, zones and boxes), map (with layers) and 3D models (with skeletons) take the place of the calling cartridge's. Returns `{title, author, res, lua, sheet_w, sheet_h, map_w, map_h, layers, [palette]}`
`cart_sheet([w, h])`	the size of the project's sprite sheet; with `w` and `h` (multiples of 8, from 8 to 4096) it resizes it: pixels that fit stay, new ones are transparent, flags stay with their tile

Function	Description
`cart_new()`	an empty sprite sheet and map (256x256, one layer), no flags, zones, boxes or models
`cart_save(path, {title, author, res, lua})`	writes a `.bm` with the code given and the current sheet, flags, zones, boxes, map, layers, cover, sound bank, models and skeletons. An 8.3 name, for example `"/bme/GAME.BME"`. Returns `true` (and the file written when it is not `path`), or `false` and a message
`cart_build(project)`	builds the game of a project (`.bme`) into a `.BM` in the games folder (next to it for a project that is not in `/bme`). Returns `true` and the game's path, or `false` and a message
`cart_read(path)`	the code and header of a `.bm`: `{title, author, res, lua, size}`, without touching the caller's sheet and map. For editors with several files open
`cart_write(path, {[lua, title, author, res, from, sections]})`	changes only the code (and the data fields) of a `.bm`; sheet, map, cover, sound bank and unknown sections stay. Without `lua` the code stays. `from`: the other sections come from another file ("save as"); `from = false`: a new cartridge. `sections`: `{[8] = MESH bytes, [9] = ANIM bytes}` (`false` takes them away), checked first. `sheet = true`: the project's sprite sheet replaces the file's, as SHEET8 when it has at most 256 colours, with the colours of `palette` (`{0xRRGGBB, ...}`) first. Returns like `cart_save`
`cart_data(kind, [bytes])`	the project's MESH (`kind` 8), ANIM (9), SPRITES (11) and BOXES (14) sections as strings in the format of `src/bm/bm.h`. Without `bytes` it returns them (`nil` if none); with `bytes` it replaces them (`nil` or `""` removes them) and returns `true`, or `false` and the reason
`cart_config(key, [value])`	the text of a key of `bm/config.txt`, or `nil`. Only keys that start with the first word of the game's title in lower case and `_` (Overbit: `overbit_`); anything else is an error. With a value the key is changed and saved (`""` clears it). Values up to 127 characters, without `;` or `,`

Audio for the editors

Function	Description
`cart_audio([path])`	the sound bank of a `.bm` as a string (`false` if it has none) and its title; without a path, the bank of the running cartridge
`cart_put_audio(path, bank, [title, lua])`	puts the bank (a string; `nil` takes it away) into a `.bm`, the rest as before; if the file is not there it creates it with that title and code. Returns like `cart_save`

Function	Description
`audio_bank(bank)`	from now on this bank plays; `nil` : none
`audio_pattern(p, bpm, swing)`	a pattern in a loop (the editors' previews)
`audio_play(v, sound, note, [vol], [fx], [ms])`	a sound of the bank on a voice (the editors' previews)

3D models

Function	Description
`cart_meshes(path)`	the meshes the code of a `.bm` builds with `mesh()` , `mesh_sphere()` and `mesh_cube()` : `{ {name=, kind=, verts={x,y,z,...}, faces={a,b,c,colour,...}, [uv={...}]}, ... }` (indices from 1, colour `-1` = texture), and `nil` or the code's first error. The code runs apart, in a Lua state of its own, with an instruction limit; bm's other functions do nothing. For bm Mesh
`mesh_reduce(record, triangles, [bones, [max_err]])`	fewer triangles for a model of the MESH section (edge collapse with quadrics, no AI). `bones` is the bone of each vertex, `max_err` stops before a costlier collapse (0: no limit). Returns the record, the bones of its vertices (`nil` without `bones`) and the number of triangles; `nil` and a message if the record is broken. On the PC: `tools/bmreduce.py`
`picture3d(action, ...)`	a picture becomes a 3D model through an image-to-3D service. `picture3d("providers")` lists the services; `picture3d("ready", service)` gives `true` , or `false` and why (the key is missing); `picture3d("start", picture, {provider=, polycount=})` starts the job on a `.png` / `.jpg` of the SD card or an https URL and gives the job id (or `nil` and a message); `picture3d("status", id, service)` gives `"running", progress` or `"done", url` ; `picture3d("take", url, {name=, faces=, height=})` downloads the `.glb` and converts it to `{record=, flat=, texture=, nv=, nf=, textured=}` . The first service is Meshy, with the key `meshy_key=...` in `bm/config.txt` . The calls block while the network works
`cutout3d(picture, {name=, lathe=, height=, depth=, segments=, faces=})`	a model from the picture's outline, made on the console without network or AI: a cutout `depth` thick, or with `lathe = true` the half outline turned round the vertical axis in `segments` steps. `height` in blocks (2), `faces` triangles at most (1200). Returns the same table as `picture3d("take")` , or `nil` and a message

Running, debugging and moving between tools

Function	Description
`cart_arg()`	the table a tool was opened with, or `nil` : `path` (the file to work on), `error` (the error of the game just tried), `back` (true when it comes back from a game), `from` (the name of the tool that called it), `run` (the dev kit's numbers of that game); `cart_run` and `cart_tool` set it
`cart_run(path, [options])`	leaves, plays that file and then opens again the cartridge that asked, with `cart_arg() = {path=, error=, back=true, run=}` . `run` holds the numbers of the run: `{frames, secs, fps, ms, ms_max, slow, lua_kb, lua_peak_kb, data_kb, instr_max, tokens, tris, gpu}`
`cart_run(path, {breaks = {lines}, stop = true})`	a session of the debugger: the game stops at the lines `breaks` of its `main.lua` , at `breakpoint()` and, with `stop` , at its first line. Stopped: F10 or A the next line, F8 or X into the call, Shift+F8 or Y out of the function, F5 or Start goes on, Esc or Select stops the game
`breakpoint([why])`	in a game tried from a tool, the debugger stops here, with the reason in its title; in other games it does nothing
`cart_tool(name, [path])`	leaves and opens another tool on the same file: `"studio"` , `"animator"` , `"mesh"` , `"pixel"` , `"code"` , `"sdk"` , `"sound"` . The tool finds it in `cart_arg()` , with `from` = the name of the tool that opened it

The assistant

A small AI that runs on the console. It understands a question (Italian or English, also with typos) and answers with entries of its knowledge base, or draws the base of a sprite. It is not a chatbot: a tiny INT8 network chooses among the entries it knows. It does nothing until you call it. Try it from **Dev > Assistant**.

Function	Description
`ai.ask(question, [{n=5, ctx=word, kinds="api,howto"}])`	the best entries, the first the most likely: `{ {id=, title=, kind=, score=}, ... }` , and as second value the microseconds taken. `ctx` : the word under the cursor. `kinds` : `api` , `howto` , `error` , `tip` , `sprite`
`ai.entry(id)`	an entry: `{id, kind, title, name, text, code, gen, see = {id, ...}}`
`ai.list([kinds])`	all the entries `{id, title, kind}`
`ai.lang([language])`	the language of the answers: `"it"` or `"en"` , and whether it follows the questions (`true`). `"it"` / `"en"` fix it, `"auto"` follows each question. It starts from `assist_lang` in `bm/config.txt` (`it` , `en` , `auto`)

Function	Description
`ai.near(word)`	the name of the API closest to a misspelt word (`"sprr"` gives `"spr"` , 1), or `nil`
`ai.sprite(request, [{gen=, size=16, seed=1, outline=true, palette={...}}])`	the base of a sprite: `{w, h, gen, name, seed, px = {0xRRGGBB or -1, ...}}` , row by row. Words choose the recipe, the colours and the size; another `seed` is a variant; with `palette` each pixel becomes the closest colour of the palette
`ai.recipes()`	the sprites' recipes `{id, name}` ; `ai.recipes("mesh")` the 3D ones `{id, name, rigged}`
`ai.script(text)`	a model written in the parts language (`mat` , `box` , `bx` , `tube` , `cyl` , `ell` , `prism` , `wedge` , `tf` , `bone` , `use` , `side` , `mirror` , `clip` , `key` , `turn` , `shift` , one per line): the same table as `ai.mesh` , or `nil` and the error (`"line 3: ..."`)
`ai.music(request, [{gen=, seed=1, key=, minor=, bpm=, bars=, inst=, context=}])`	music for a sound bank: rhythms, backing tracks, bass lines, arpeggios, melodies and sound effects, 87 recipes. Words choose the recipe, the key, the tempo, the length and the instrument. `context = {notes = {...}, bars = {...}}` : the notes already there (MIDI) and the bar of each. Returns `{gen, name, kind, bpm, swing, key, minor, meter, bars, echo, room, seed, chords, instruments, patterns, sfx}` . On the RGB30 too, where the words choose the recipe without the network. `ai.music_recipes()` lists them
`ai.mesh(request, [{gen=, seed=1, scale=1, rig=true}])`	the base of a 3D model for bm Studio and bm Animator: `{gen, name, seed, faces, bones, clips}` , the same tables as `bm3d.lua` . 53 recipes (shapes, objects, people, animals, machines); `"no skeleton"` in the words leaves out the skeleton
`ai.checksum(question)`	the CRC-32 of the network's outputs for a question: for the tests

Small networks for games

A network of dense layers with INT8 weights and activations, from a string a training script writes (`scripts/nnetlib.py`).

Function	What it does
`nnet(blob)`	the network of a "BMNN" string (format in `src/ai/net.h` ; an error if broken), up to 8 layers of 256
`net:run(inputs, [outputs])`	the outputs for a table of inputs; `outputs` : a table to fill instead of a new one

Function	What it does
`k, v = net:pick(inputs, [mask])`	the index (from 1) of the biggest output and its value; `mask` : a table of booleans, `false` = that output is not chosen
`n_in, n_out = net:size()`	how many inputs and outputs

The assistant panel

`require "assist"` gives the panel the tools open with a key (F6 in the Assistant). It answers while you type; Enter (A) passes the code or sprite to the tool, Esc (B) closes, Tab (X) changes mode.

```
local assist = require "assist"

function _update()
  if assist.update() then return end          -- open: the keys are its own
  local k = keyp()
  if k == "f6" then
    assist.open{ mode = "code", ctx = word_under_cursor,
                 on_insert = function(code) insert_lines(code) end }
  end
end

function _draw()
  draw_tool()
  assist.draw()                               -- on top, if open
end
```

Function	Description
`assist.open{...}`	opens the panel. `mode` : `"code"` , `"sprite"` , `"mesh"` , `"error"` or `"any"` ; `query` , `ctx` , `error` , `size` and `palette` (sprites), `on_insert(code)` , `on_sprite(sprite)` , `on_mesh(m)` , `on_close()` , `x, y, w, h` (default: almost the whole screen)
`assist.update()`	every frame; true while the panel is open and uses the keys
`assist.draw()`	every frame, on top
`assist.is_open()` / `assist.close()`	asks / closes
`assist.act(request, lines, n)`	what a line `#entry: request #` at line `n` of `lines` asks: ternary operator, comment, log, remove the logs, nil check, make local, indentation, rename, optimise, explain, an example to insert or a sprite written as code. Returns `{lines, ok, message, cursor, explain}` ; changes nothing if it is not sure enough

Word completion and typing with the pad

Function	Description
`predict.complete(text_before_the_cursor, {lang = "lua"})`	`nil` or `{prefix, word, rest, ending, list}` : the most likely word that starts as the one written. `lang` : `"it"` , `"en"` , `"lua"` , `"ask"` , a mix with weights (`{it = 1, ask = 2}`) or `"none"` ; `words` the code's names
`predict.count_words(lines)`	the names of the code, for `words`
`predict.preload({"lua", "it"})`	reads the dictionaries one piece per frame, instead of at the first word
`pt.update(host)`	every frame; reads `pad()` and changes the text through the host (`before` , `insert` , `erase` , `newline` , `move` , `lang`)
`pt.text_host(lang)`	makes a host on a string
`pt.draw(x, y)` / `pt.size()`	draws the controller's overlay / its size
`pt.coach(host, text)`	the next button to write a text

`predict` is described in `docs/PREDICT.md` and `padtype` in `docs/PADTYPE.md` .

Resource files and sharing assets

A game's resources live inside the sections of its `.bm` . A resource file carries one resource outside a `.bm` , so you can move it to another project.

The file types

Extension	Resource	Sections
`.bmm`	3D models	MESH, ANIM (if animated), SHEET8 or SHEET (only the cells the textures use)
`.bmi`	images and sprites	SHEET8 or SHEET, SPRITES, BOXES, FLAGS
`.bms`	sounds	AUDIO
`.bmt`	tile map	MAP, LAYERS, FLAGS, SHEET8 or SHEET (only the tiles the map uses)
`.bmc`	palette	SHEET8 of N x 1 pixels (N up to 256)
`.bmk`	kit: several types together, one shared sheet	any combination of the sections above

What it does

A resource file uses the same container as a `.bm` , with the magic `BMRES` and three zero bytes instead of `BMCART` , and never a LUA section. The type at offset 12 is 1 for `.bmm` , 2 `.bmi` , 3 `.bms` , 4 `.bmt` , 5 `.bmc` , 6 `.bmk` . The type counts more than the extension: a renamed file is still recognised. Every file can carry an INFO section.

Good to know

- Names are 8.3: the extension is `bm` plus one letter (never `.bmp`). The real name is inside the file.
- The menu does not list resource files among the games, and games do not read them while running. A resource enters a `.bm` by being copied in; the cartridge stays one file that works on its own.
- INFO is text, `key: value` lines: `name` , `desc` , `author` , `license` (an SPDX id such as `CC0-1.0`), `version` , `tags` (separated by commas), `origin` (the SHA-256 of the resource it was copied from), `date` (YYYY-MM-DD). A block `[type name]` gives the keys of one part; types are `model` , `sprite` , `sound` , `sfx` , `song` , `map` , `palette` . At most 16 KiB; unknown keys are kept.
- A sprite zone moves in the sheet whole. In a game, `zspr(name, x, y)` draws it and `zboxes(name, [frame, type])` reads its boxes.
- The limits are those of the `.bm` : a sheet up to 4096x4096, 256 models, 64 bones.

Caveats

- The Lib tab, which lists the resources, is hidden for now. `lib_tab=1` in `bm/config.txt` brings it back.

- The apps that open resource files are not done yet: A in the Lib tab opens only resources that are inside a `.bm`.
- `bmres.py add` with a `.bme` cart and no `-o` writes the `.bme` in place. With a `.bm` or `.b16` and no `-o` it refuses to change the game ("is a game, read only") and writes a copy with the same name and the `.bme` extension. A `.bme` output is marked as a project.
- [TO FILL: `docs/RISORSE.md` says `mkbm.py --res FILE` integrates resources at build time, but in `scripts/mkbm.py` the option `--res` chooses the screen resolution. Say which tool does the build-time integration.]

What integration does

When you add a resource file to a `.bm`, `bmres.py` follows these rules. It never removes anything from the destination.

- Textures, sprites and tiles take free 8x8 cells of the destination sheet. A cell is free if it is transparent and nothing uses it. Cell 0 is never used.
- If there is no room, the sheet grows in height, up to 4096. The width stays, so sprite and map numbers do not change. An island wider than the sheet does not fit, and a message says so.
- The texture corners, the SPRITES zones and the map numbers are then corrected.
- Palettes are merged. Beyond 256 colours the sheet becomes SHEET by itself.
- If a name already exists a number is added (16 characters for models and zones, 8 for sounds).
- A skeleton follows its model.
- Sounds go into the free slots.
- The INFO of the resource goes into the entry's block, with `origin`.

Try it, step by step: move a model from one game to another

You need Python 3 and the repository. The commands run on the PC, from the repository folder.

1. See what the source game holds.
2. Extract the model into a `.bmm`. Use the name `list` showed. You get `VILLAGER.bmm`, with its skeleton and only the texture cells it uses.
3. Write who made it and under which licence. The file keeps this through every copy.
4. Add it to the other game. Use `-o` to write a new file and keep the original.
5. Run `list` on the result. The model is there.

COPY AND PASTE

```
python3 scripts/bmres.py list GAME.bm
python3 scripts/bmres.py extract GAME.bm -o VILLAGER.bmm --model villager
python3 scripts/bmres.py info VILLAGER.bmm --name "Villager" --author me --license CC0-1.0 --tags charac
python3 scripts/bmres.py add OTHER.bm VILLAGER.bmm -o OUT.bm
python3 scripts/bmres.py list OUT.bm
```

TRY IT

Extract the same model twice and add both files to one game. The second gets a number added to its name.

BEWARE

If `add` says an island does not fit, the destination sheet is narrower than the pieces. It does not widen the sheet.

The other commands

The kind of a new resource file comes from the extension of its name. The options `--name`, `--author`, `--license`, `--version`, `--tags` and `--desc` also work with `extract` and `convert`.

COPY AND PASTE

```
python3 scripts/bmres.py extract GAME.bm -o OUT.bmi --sprite NAME
python3 scripts/bmres.py extract GAME.bm -o OUT.bmi --rect X,Y,W,H --name N --frames N --fps N
python3 scripts/bmres.py extract GAME.bm -o OUT.bms --sfx NAME --song NAME --sound NAME
python3 scripts/bmres.py extract GAME.bm -o OUT.bmt
python3 scripts/bmres.py extract GAME.bm -o OUT.bmc
python3 scripts/bmres.py extract GAME.bm -o OUT.bmk
python3 scripts/bmres.py convert IN OUT
python3 scripts/bmres.py convert IN.csv OUT.bmt --tiles TILES.png
```

`extract` with none of `--sprite` or `--rect` takes the whole sheet; with none of the sound options it takes the whole sound bank. `convert` goes between these pairs:

From	To
`.png`	`.bmi`
`.bmi`	`.png`
`.glb`	`.bmm`
`.json`	`.bms`
`.bms`	`.json`
`.bm` (a sound pack)	`.bms`
`.hex`, `.gpl`	`.bmc`
`.bmc`	`.hex`, `.gpl`
`.csv`	`.bmt` (`--tiles`: the sheet its numbers refer to)
`.bmt`	`.csv` (and the tiles `.png`)

The project layout

A game or an app lives in `carts/<id>/`. The common layout keeps the metadata in one file, `bm.info`. A folder without `bm.info` still builds the old way, with title, author and resolution from the Makefile, so you can move one game at a time.

```
carts/<id>/
  bm.info        metadata (INFO section), the single source
  main.lua       entry point (always this name)
  cover.png      menu cover (optional)
  assets/        sheet.png, map.csv, map_<layer>.csv, flags.csv, sprites.txt, models.bm|glb
  res/           shared resource files .bmm .bmi .bms .bmt .bmc .bmk
  build/         the game's own build scripts (mk*.py, build.py)
```

What it does

`bm.info` has the same syntax as the INFO section: `key: value` lines, `[type name]` blocks for parts, and `#` lines that are comments and are not copied. `mkbm.py --info` copies it into the cartridge.

Key	Meaning	Used by the build
`name`	name shown	default of `--title`
`author`	author	default of `--author`
`resolution`	`640x360`, `480x270`, `320x180`, `256x256`, `360x360`, `720x720`	default of `--res`
`desc`, `license`, `version`, `tags`, `date`	as in the INFO section	stored only

Good to know

- The Makefile looks in `assets/` first, then in `carts/<id>/`, so old games still work.
- Unknown keys are kept in the cartridge. The binary `.bm` format does not change, and an older kernel ignores INFO.
- Games already moved: `pong` (`bm.info`), `yharnam` (`bm.info`, `sheet.png` and `map_*.csv` in `assets/`, the `mk*.py` scripts, `mapview.py` and `art/` in `build/`) and `hunt` (`bm.info`, `sheet.png` and `map.csv` in `assets/`, `mkassets.py` in `build/`).

Caveats

- `--info` goes with `--lua`; you cannot use it with `--from`.
- Moving files into `assets/` and `build/` changes the paths in the game's own `mk*.py` and in any doc that names them.
- The `.bm` of a migrated game gains an INFO section of a few hundred bytes. Otherwise it is identical.
- Its `title_<id>` line leaves the Makefile.

Try it, step by step: migrate a game to the common layout

The example moves a game called `ball`, with a sheet and a map.

1. Create `carts/ball/bm.info` with the name, author and resolution of the game (see the block below).
2. Create the folder `carts/ball/assets/` and move `sheet.png` and `map.csv` into it.
3. If the game has scripts of its own (`mk*.py`), create `carts/ball/build/` and move them there. Open each one and fix the paths to `main.lua` and `assets/` .
4. Remove the game's `title_ball` line from the Makefile, because the title now comes from `bm.info` .
5. Build the game.
6. Compare with the `.bm` you built before the move. It must be identical except for the INFO section.

COPY AND PASTE

```
name: Ball
author: me
resolution: 640x360
license: CC0-1.0
version: 1
tags: demo
```

COPY AND PASTE

```
python3 scripts/mkbm.py -o ball.bm --lua carts/ball/main.lua --sheet carts/ball/assets/sheet.png --map c
```

BEWARE

If the build cannot find a file after you moved it, a path in one of your own scripts is still the old one. Fix the path; do not copy the file back.

Screenshots for developers

A game or an app saves the frame it is drawing as a PNG on the SD card. The player does it with a key; your code does it with `shot()`.

How	Where
Print Screen on the keyboard	`bm/shots/` on the SD card
Select held for 3 seconds on a pad (Share on a DS4)	`bm/shots/` on the SD card
`shot([prefix])` in your code	`bm/shots/` on the SD card

What it does

A box says `Screenshot saved` and the file name for two seconds. The picture is the game's own screen as it was drawn: the dev kit's overlays, the battery icon, the pointer and that box are not in it. Games and apps run by bm's menu take it; the menu's own screens do not yet.

Good to know

- `shot([prefix])` asks for the frame being drawn. It is saved when `_draw` ends, so call it at any point of the frame; calling it twice in a frame gives one file.
- It returns the name the file will have, or `nil` and a message if the numbers are over. It asks the player nothing.
- The name is `<PREFIX><NNNN>.PNG`, 8.3. The prefix is the first, up to 4, letters or digits of the game's title (`Pong` gives `PONG`, `Hunter's Night` gives `HUNT`, `SHOT` if there are none) or the `prefix` you gave.
- The number is one more than the highest of that prefix already in `bm/shots/`, from 0001 to 9999. The Pi Zero has no clock, so there is no date.
- The format is PNG, 8 bits RGB, the size of the game's screen (640x360, 360x360...). Nothing is lost: the console's 16-bit colours are widened to 8 bits. There is no JPG, because it spoils pixel art.
- From the PC, over the network link, `python tools/bm_net.py IP --shot out.png` asks the console for a screenshot of the running game (the console asks for the Console password) and saves it as `out.png`. `python tools/bm_net.py IP --get bm/shots/PONG0007.PNG out.png` fetches a file that is already in `bm/shots/`. A game must be drawing frames: with none, the request fails after 3 seconds. Only files of `bm/shots/` can be fetched. This is not verified on an RGB30.
- From the PC, `python tools/bm_net.py IP --play game.bm` sends a `.bm` to the console and plays it at once. The console asks for the Console password. The file is not saved on the SD card. The menu starts the game when it has the keys; if an update or a Market publish is running, it shows a message and does not start the game. `--send game.bm` saves the file on the SD card instead. This is not verified on an RGB30.
- A flat screen takes a few KB; noise can take as much as the raw picture (about 0.7 MB at 640x360).

Caveats

- [TO FILL: check Select held 3 s on the PowKiddy RGB30, and `shot()` with a 3D game on the GPU; the repository says neither was tried on a Pi or an RGB30.]

Try it, step by step

1. Add the lines below to the `_draw` of a game of yours and pack it.
2. Run the game on the console and press **A** (or the key your game uses).
3. Wait for the box that says `Screenshot saved`.
4. Put the SD card in your computer and open the folder `bm/shots/`. The picture is there.

[TO FILL: shots/api-tools-shot-saved.png]

TRY IT

Call `shot("menu")` and `shot()` in the same game. The first file starts with `MENU`, the second with the first letters of the title.

COPY AND PASTE

```
local path = shot()          -- "bm/shots/PONG0007.png"
local path = shot("menu")    -- "bm/shots/MENU0001.png"
```

For a manual

Pictures for a manual are taken on the console, copied from `bm/shots/` to the repository and given a name that says what they show. On the PC, `bmhost --shots` and `scripts/qemu-screenshot.sh` make pictures without a console, and `make test-shot` checks the PNGs.

Open points

- **The cartridge file**: status of the `.b16` profile. `docs/B16.md` says the profile is decided but not yet implemented, and §3 describes a different `B16CART` container that §0 replaces with the `.bm` container; `src/bm/project.h` and `scripts/mkbm.py` already mark and export `.b16` files (bit 1 at offset 18, 8 MiB check). Say which one is current.
- **The cartridge file**: picture to take: shots/api-tools-editable-copy.png
- **Resource files and sharing assets**: `docs/RISORSE.md` says `mkbm.py --res FILE` integrates resources at build time, but in `scripts/mkbm.py` the option `--res` chooses the screen resolution. Say which tool does the build-time integration.
- **Screenshots for developers**: check Select held 3 s on the PowKiddy RGB30, and `shot()` with a 3D game on the GPU; the repository says neither was tried on a Pi or an RGB30.
- **Screenshots for developers**: picture to take: shots/api-tools-shot-saved.png